

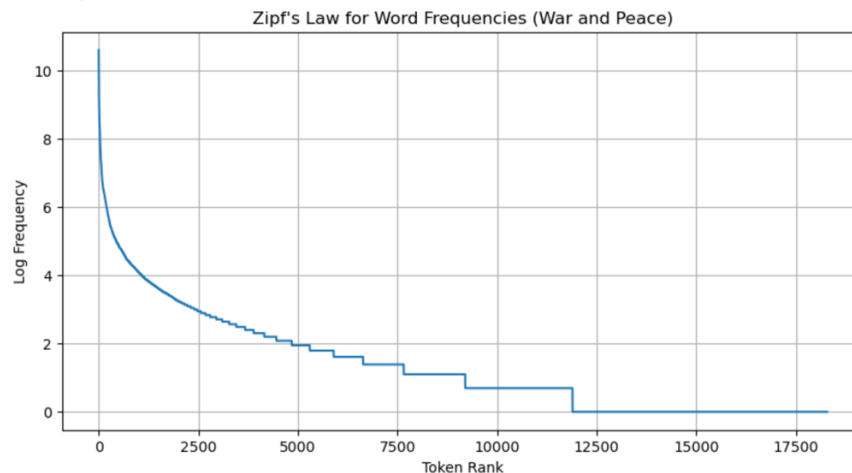
Name: Sreeja Reddy Yeluru

Development environment

- For this assignment, I worked on MacOS (Sonoma 14.4.1) and used Anaconda Navigator to launch Jupyter Notebook (local) for writing and testing my code. All of the code was written and run in Python 3.x (Anaconda base environment). Everything was done in a local development setup.
- The main libraries and packages I used were:
 - re for regular expressions
 - string for checking punctuation
 - collections.Counter for token frequency counts
 - math and matplotlib.pyplot for the Zipf's Law plot
 - nltk.tokenize.word_tokenize and nltk.tokenize.sent_tokenize for token and sentence splitting

Zipf's Law Chart (Task #2)

I applied my code to war-and-peace.txt and plotted a Zipf's law curve using the top tokens.



The graph plotted clearly shows a steep downward slope where the most frequent words (like "the", "and") appear thousands of times, which is expected. A sudden drop in frequency for words ranked lower. Thereby, the pattern confirms Zipf's Law. Word frequency is inversely proportional to its rank. Though some of my counts didn't match straight up with the output but the overall frequency distribution and chart shape are correct. Which confirms the logic for tokenisation and frequency sorting worked correctly.

General Reflections

- For task 1, my output matched exactly with the sample output. The formatting is correct. There is a two-space gap before the frequency counts. The sorting is also correct. It is sorted by frequency in descending order and lexicographically for ties.

For task 2, my output was very close but did not match in terms of the number of sentences, tokens, and unique tokens. I found that differences were caused as I used `nltk.sent_tokenize()` and `nltk.word_tokenize()` for sentence and word splitting. These functions don't always match the sample output on a huge text like `war-and-peace.txt`. NLTK sometimes either splits or merges sentences differently due to quotes, abbreviations, or punctuation patterns. I tried fixing some edge cases, like quoted sentences, using NLTK, which brings out slight mismatches in sentence splitting.

- This assignment helped me in understanding that tokenisation is not a one that fits all. A small punctuation mark can throw off sentence splitting and change everything. Also, how contractions like "shouldn't've" need to be expanded fully and not just partially. Making rules that work on both small files and huge texts was a big challenge.
- Conceptually, this assignment is not super hard, but it required a lot of careful debugging. I spent a lot of time comparing my token counts to the sample outputs line by line to understand why something is off.

- I went through a couple of difficulties with matching exact token counts when NLTK splits something unexpectedly. To decide when to override NLTK's logic and when not to, handling double contractions like "shouldn't've" properly, sorting tokens exactly as required.
- If I had more time, I would have spent more time analysing where `nltk.sent_tokenize()` goes wrong and try to write a few more fixes specifically for task 2.

Conclusion

Overall, I feel like I implemented everything the assignment asked for, and I understood every piece of the logic I wrote. I enjoyed the debugging process.